UNIVERSITÀ DEGLI STUDI DI UDINE

Laboratorio di Programmazione Web
(laurea in Scienze e Tecnologie Multimediali)

A.A. 2025/2026

Web link alle pagine del corso:

http://elearning.uniud.it/moodle/course/view.php?id=5787

Client-Server communication
HTTP

Autore:

Carlo Drioli

I materiali in queste slide sono tratti in parte da:

V. Della Mea, L. Di Gaspero, I. Scagnetto,"Programmazione lato server", Apogeo.

http://www.w3.org/

Wikipedia

Client–Server communication

I. Web applications

- Web applications are interactive software programs that provide their feedback to user requests through the access to distributed resources over computer networks.

- Access to distributed resources is achieved through the client-server communication paradigm, relying on the HTTP network protocol.

II. Static vs. dynamic web sites

- A *static website* is one that has its resources (i.e. web pages) stored on the server in the format that is sent to a client web browser

- A *dynamic website* is one that is able to dynamically generate new resources upon client requests

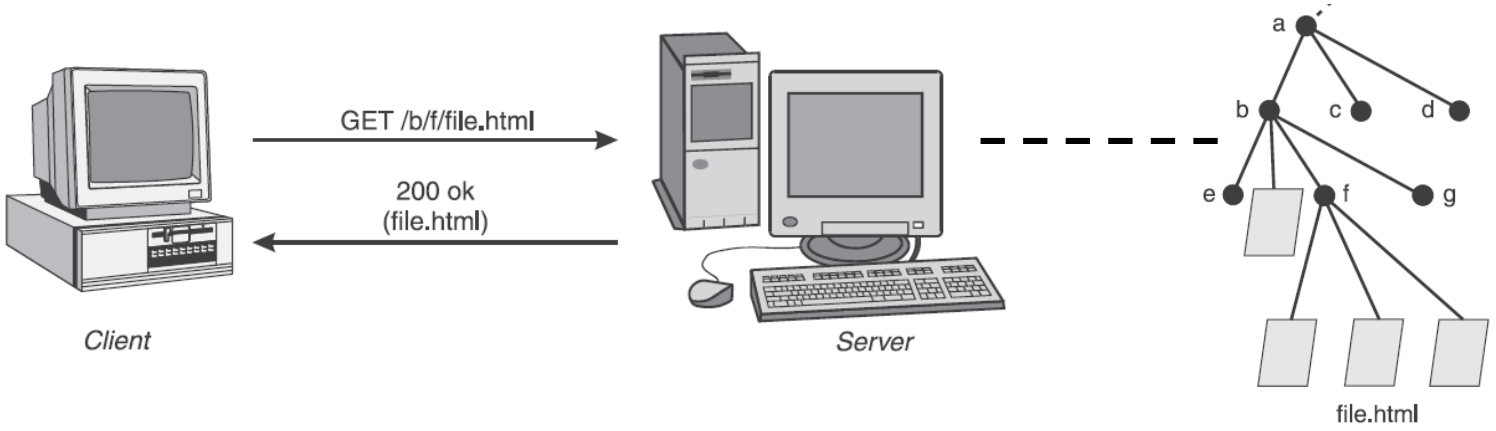
- *Server-side programming* refers to the techniques used to generate new resources in real-time, based on clients requests.

Client–Server communication

III. The web server functions

- A *web server* is a *daemon* which constantly listens on a TCP port (usually port 80) for clients requests, according to the HTTP protocol.

- Upon a client request for a web resource or service hosted by the server, a document is fetched on the file-system or generated dynamically, and sent to the client.



A static web site server

Client–Server communication

IV. The client-server HTTP request/reply scheme

- When the user asks for a new web resource, the browser (client) sends an *HTTP request* to the server
- The HTTP request is a data structure, sent to the server host, containing all the information to find/generate the resource
- Upon receipt of a HTTP request, the server sends an *HTTP reply* to the client
- The HTTP reply is a data structure containing control information and, if successful, the requested resource.

HTTP protocol

V. The HTTP request

- First line includes the *method* to be applied to the resource, the *identifier of the resource*, and the protocol version in use.

- The methods are the different commands that can be issued by the client to obtain services from the server. The most relevant ones are:

GET	ask to server for a given resource (GET *resource* HTTP/*ver*)
POST	send data to a resource on the server *(* POST *resource* HTTP*/ver)*

- Following lines include header information

HTTP protocol

V. The HTTP request: example (GET)

URI: http://www.dominio.com/dir1/dir2/file.html



```
1  GET /dir1/dir2/file.html HTTP/1.1
2  Host:www.dominio.com
3  Accept: text/xml,application/xml,application/(X)HTML+xml,text/
       html,text/plain,image/png
4  Accept-language: it,en
5  Accept-charset: ISO-8859-1
6  User-agent: Mozilla/5.0 (Macintosh; U; PPC Mac OS X Mach-O; it;
       rv:1.8.0.5) Gecko/20060723 Firefox/1.5.0.5
7  Referer: http://www.google.it/search?q=un+ottimo+libro&start=0&
       ie=utf-8
```

HTTP protocol

V. The HTTP request: POST

- The POST method sends data to a resource on the server (POST resource HTTP/ver)
- The data is embedded in the body of the request, in the form of attribute=value pairs.
- The resource is usually a server-side program able to process the data and to generate a new dinamically generated resource, that will be included in the HTTP reply

HTTP protocol

V. The HTTP request example: POST

Sample HTTP Post Request

```
POST /path/to/upload/script HTTP/1.0
Connection: Keep-Alive
User-Agent: My Client App v1.0
Host:
https://bulksell.ebay.com/ws/eBayISAPI.dll?FileExchangeUpload
Content-type: multipart/form-data;
boundary=THIS_STRING_SEPARATES
Content-Length: 256
--THIS_STRING_SEPARATES
Content-Disposition: form-data; name="token"
12345678987654321
--THIS_STRING_SEPARATES
Content-Disposition: form-data; name="file";
filename="listings.csv"
Content-Type: text/csv
... contents of listings.csv ...
 --THIS_STRING_SEPARATES-
```

HTTP protocol

VI. The HTTP reply

- The first line includes a success/failure state code, possibly followed by header information
- The following lines contains the requested resource (e.g., the HTML source)

Example

```
1  HTTP/1.1 200 OK
2  Date: Mon, 31 Jul 2006 15:27:59 GMT
3  Server: Apache/2.0.54 (Debian GNU/Linux) mod_python/3.1.3
       Python/2.3.5 PHP/4.3.10-16
4  Expires:  Thu, 19 Nov 2006 08:52:00 GMT
5  Content-Type: text/html
6  Content-Length: 3451

8  <html>
9  <head>
10 ...
11 (tutto il file richiesto)
```

User interaction in web applications

I. Data input through forms

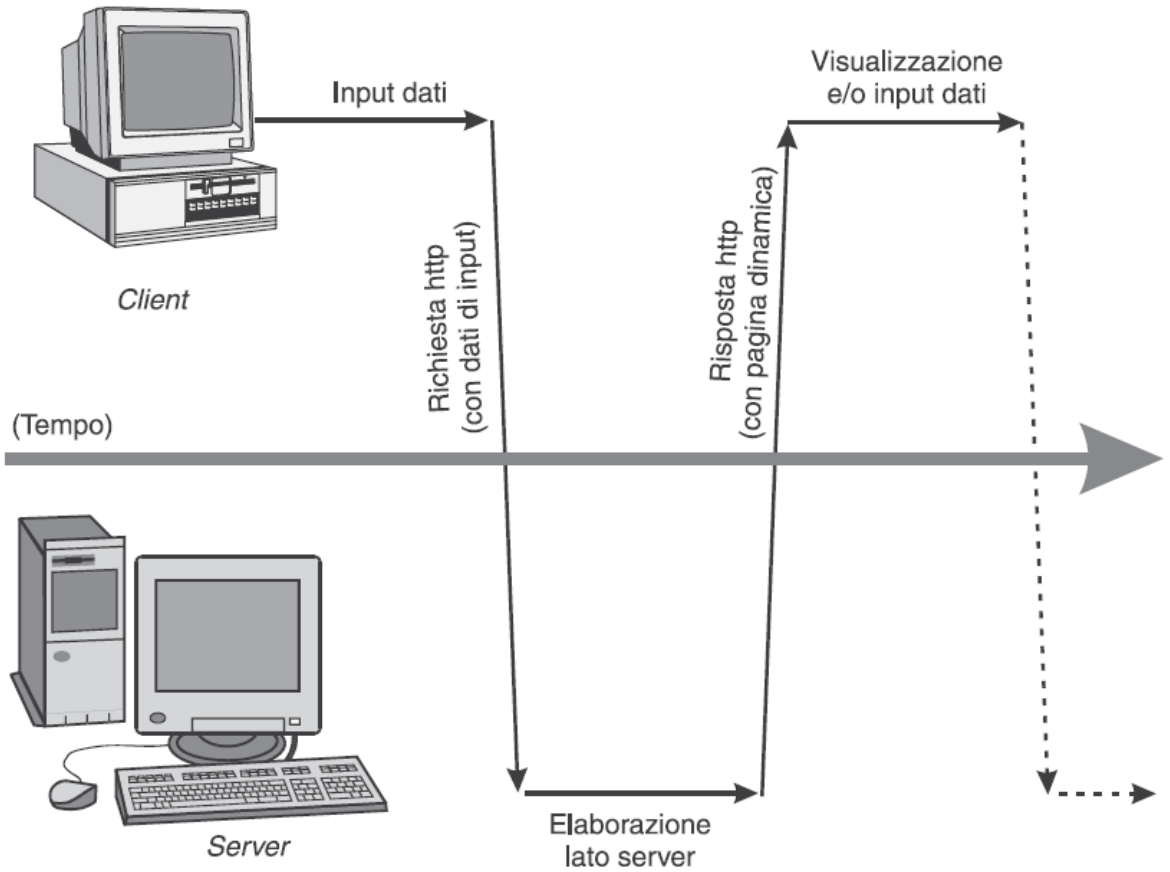
- Any computer application should provide support for processing user-generated data
- Web applications provide an user interaction mechanism through *input forms*:

Form interaction scheme

- Data from user in input fields is embedded in the HTTP request and sent to a server-side processing resource
- The web server transmit the data to the data processing resource
- The processing resource generates a new web page which is sent to the client
- The client (browser) displays the web page received

User interaction in web applications

I. Data input through forms



User interaction in web applications

II. (X)HTML forms

The form element syntax includes attributes to specify data sending modes (*method*) and remote processing resource (*action*)

```
1 <form method="post" action="http://www.dominio.com/URI_azione">
2  <p>
3   <input type="text" name="campo1" /><br />
4   <input type="text" name="campo2" /><br />
5   <input type="submit" name="Invia" />
6  </p>
7 </form>
```

III. GET/POST methods

The *method* attribute can assume two values: “get” or “post”, and are used to chose between two different data sending methods.

User interaction in web applications

III. GET/POST methods

- The GET method: the input data are appended to the URI included in the first header line of the HTTP request.

```
1  GET /URI_Azione?campo1=valore1&campo2=valore2 HTTP/1.1
2  Host:www.dominio.com
```

- The POST method: the input data are included in the HTTP request body:

```
1  POST  /URI_Azione HTTP/1.1
2  Host:www.dominio.com
3  Content-type: application/x-www-form-urlencoded
4  Content-length: 31
5  campo1=valore1
6  &campo2=valore2
```

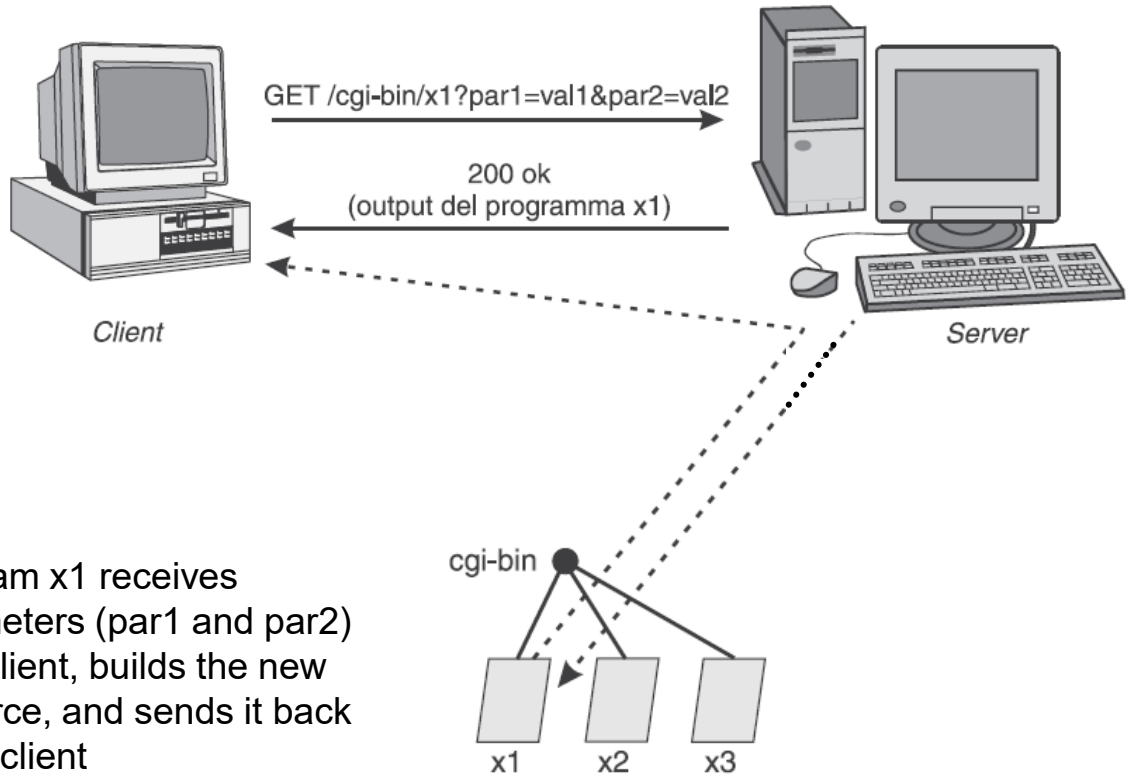
User interaction in web applications

III. GET/POST methods

- The GET method:
  - header space is limited, thus only a limited amount of information can be sent with this method
  - sent data is visible to the user. This can be a problem for personal or sensible data
- The POST method:
  - data are transmitted in the HTTP body and is not immediatly visible to the user
  - the HTTP body has no size limits, and the POST method can be used to transmit large amount of data

Server-side technologies

I. Common gateway Interface (CGI)



Program x1 receives parameters (par1 and par2) from client, builds the new resource, and sends it back to the client

Server-side technologies

I. Server-side scripting

- The resource requested by the client is typically an (X)HTML document with script fragments.
- Script fragments can be included in (X)HTML documents usign scripting tags <? ?>
- A PHP script example:

```
1  <html>
2   <head>
3    <title>Esempio 1</title>
4   </head>
5   <body>
6    <h1>Esempio con PHP</h1>
7    <p>
8    <?php
9     echo "questo &egrave; un semplice script";
10   ?>
11   </p>
12   <?php
13     echo "<p>ed anche questo!</p>";
14   ?>
15   </body>
16  </html>
```

I. Server-side scripting

- The resource after the script code interpretation:

```
1  <html>
2   <head>
3    <title>Esempio 1</title>
4   </head>
5   <body>
6    <h1>Esempio con PHP</h1>
7    <p>
8     questo &egrave; un semplice script
9    </p>
10   <p>ed anche questo!</p>
11  </body>
12 </html>
```

- This is the actual html document that wil be sent to the client

Server-side technologies

I. Server-side scripting

- The PHP script fragment interpretation process:

